# HTML pages and tables

Study guide 7: build an HTML page from scratch, join it to your Django project, and show data in tables you can filter, sort, page and download

## What you'll build

1. **A plain HTML page from scratch**, a header, a heading and a styled table, that opens straight in your browser. No Django, no Python: just HTML and CSS, so you see what they are on their own.
2. **A real page in your project, the Ledger explorer**: pick a month, a client and a ledger type, and see the ledger rows in a table you can sort by clicking a heading, page through 50 rows at a time, and download as CSV.

## How to use it

- Parts 2 to 5 build up one small step at a time. Type each step yourself and check it works before the next one.
- The finished code is in nexora_reports_django.zip (the explorer app) and hello_table.html.
- Part 7's checkpoints are real tests from the project; all 35 project tests pass.

# Part 1. Think before you write code

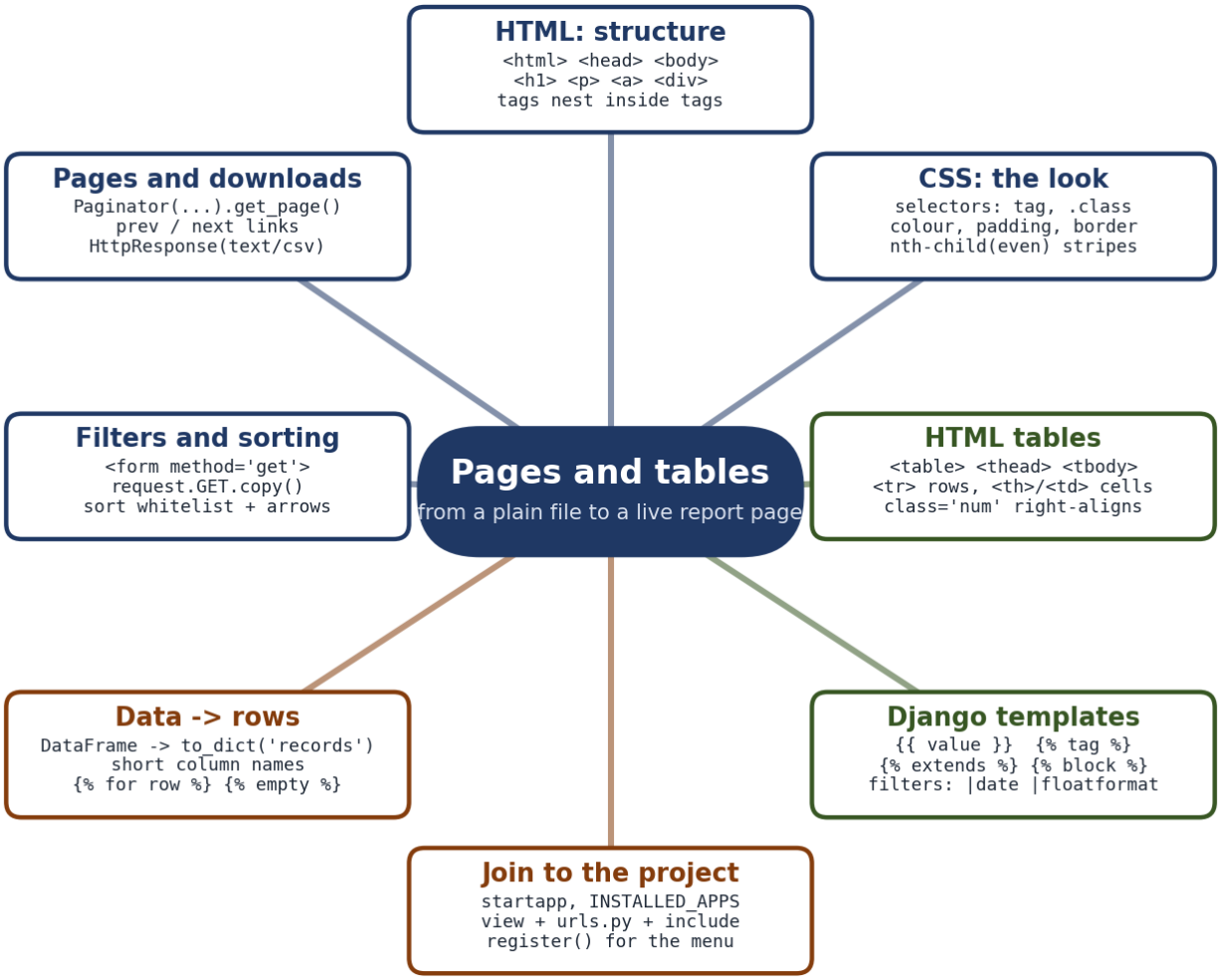
A web page is three layers. Keeping them apart in your head makes every page easier to build:

| Layer | Job | Written in | Example |
|---|---|---|---|
| Structure | What is on the page, and in what order | HTML | `<h1>, <table>, <form>` |
| Look | Colours, spacing, borders, alignment | CSS | `.num { text-align: right; }` |
| Data | The numbers and names, different every time | Django template + Python view | `{{ row.debit\|floatformat:'2g' }}` |

## The plan for any new page

1. **Draw it**: which inputs, which numbers, which columns.
2. **Write it as plain HTML** with made-up rows, until it looks right.
3. **Move it into a Django template** and replace the made-up rows with a loop.
4. **Write the view** that prepares exactly the data the template loops over.
5. **Add the URL and the menu link**, then tests.

## Everything in this guide as a mindmap



# Part 2. HTML from scratch (no Django)

## Step 1. The smallest page

> **What I was thinking**
> "A browser needs very little: a line saying this is HTML, a head for settings, and a body for what to show."
> "Tags come in pairs:  opens,  closes. Whatever sits between them is inside that element."

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>My first table</title>
</head>
<body>
  <h1>Broker summary</h1>
  <p>August 2026. Numbers are in USD.</p>
</body>
</html>
```




| Tool | What it does | Why here |
|---|---|---|
| `<!doctype html>` | Tells the browser it's modern HTML | Always the first line |
| `<html lang='en'>` | The whole page | lang helps screen readers and translation |
| `<head>` | Settings: title, character set, styles | Not shown on the page |
| `<meta charset='utf-8'>` | Use UTF-8 text | Arabic, Hindi and symbols like ▲ display correctly |
| `<title>` | The browser tab's text | Also the bookmark name |
| `<body>` | Everything you see | All visible content goes here |
| `<h1> ... <h6>` | Headings, biggest to smallest | One h1 per page |
| `<p>` | A paragraph | Text blocks |

**Check yourself**
Save it as hello.html anywhere, double-click it: your browser shows the heading and the paragraph.

## Step 2. Sections and links

**What I was thinking**
"Pages have regions: a header bar, the main content. Named tags say what each region is."
"div is a plain box with no meaning of its own; I'll give it a class so CSS can style it."

```
<body>
  <header>
    <strong>Nexora Reports</strong>
  </header>
  <main>
    <h1>Broker summary</h1>
    <p>August 2026. Numbers are in USD.</p>
    <div class="card">
      ... the table goes here (Step 4) ...
    </div>
    <p><a href="https://developer.mozilla.org/en-US/docs/Web/HTML/Element/table">Read more about HTML tables</a></p>
  </main>
</body>
```

| Tool | What it does | Why here |
|---|---|---|
| `<header>, <main>` | Named regions | Clear structure; CSS can target them |
| `<div class='card'>` | A plain box with a class name | Reuse one style for many boxes |
| `<strong>` | Bold, important text | The site name |
| `<a href='...'>text</a>` | A link | href is where it goes |
| `attribute='value'` | Extra information inside a tag | class, href, lang are attributes |

**Check yourself**
Click the link: it opens the page in href.

## Step 3. CSS: how it looks

**What I was thinking**
"HTML says what things are; CSS says how they look. Rules go in a <style> block inside <head>."
"A rule is: which elements (the selector), then { property: value; }. A tag name selects every tag of that kind; a dot selects a class."

```
<style>
  body   { font-family: Arial, sans-serif; color: #1f2937; background: #f7f8fa; margin: 0; }
  header { background: #1f3864; color: white; padding: 14px 24px; }
  main   { max-width: 900px; margin: 24px auto; padding: 0 24px; }

  /* .card = every element with class="card" */
  .card  { background: white; border: 1px solid #e5e7eb; border-radius: 8px; padding: 20px; }
</style>
```

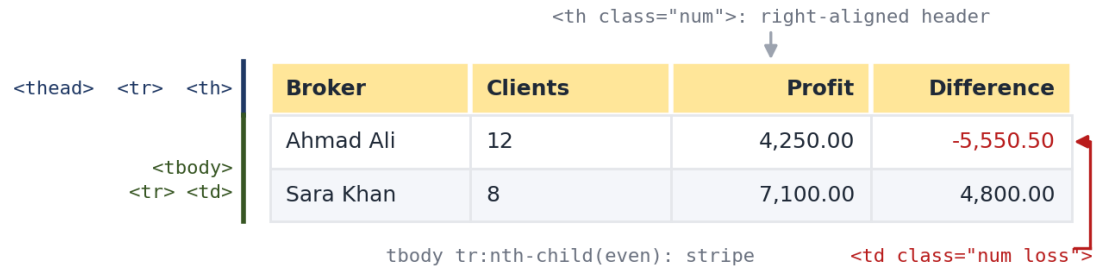
| Tool | What it does | Why here |
|---|---|---|
| `body { ... }` | A rule for every <body> | Font and colour for the whole page |
| `.card { ... }` | A rule for class='card' | One look, reused everywhere |
| `color / background` | Text colour / fill colour | Hex codes: #1f3864 is your navy |
| `padding / margin` | Space inside / outside the border | padding: 14px 24px = top-bottom, left-right |
| `margin: 24px auto` | Equal space left and right | Centres the main column |
| `border-radius` | Rounded corners | Softer boxes |
| `/* ... */` | A comment in CSS | Notes for you, ignored by the browser |

**Check yourself**
Change the header's background to #375623 and reload: the bar turns green.

## Step 4. A table

**What I was thinking**
"A table is rows of cells. Header cells (th) go in thead; data cells (td) go in tbody, one tr per row."
"Numbers read best right-aligned, so their columns get class='num'."

```
<table>
  <thead>
    <tr>
      <th>Broker</th> <th>Clients</th> <th class="num">Profit</th> <th class="num">Difference</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ahmad Ali</td> <td>12</td> <td class="num">4,250.00</td> <td class="num loss">-5,550.50</td>
    </tr>
    <tr>
      <td>Sara Khan</td> <td>8</td> <td class="num">7,100.00</td> <td class="num">4,800.00</td>
    </tr>
  </tbody>
</table>
```



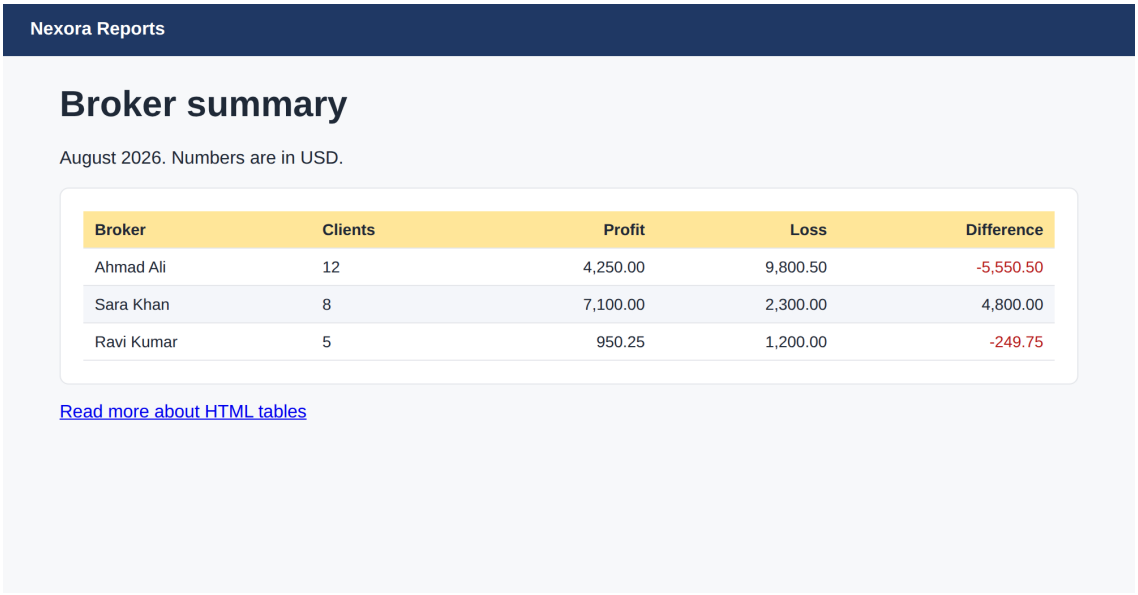
| Tool | What it does | Why here |
|---|---|---|
| `<table>` | The whole table | Holds thead and tbody |
| `<thead> / <tbody>` | Header rows / data rows | CSS can style them separately |
| `<tr>` | One row | Contains th or td cells |
| `<th> / <td>` | Header cell / data cell | Same number per row |
| `class='num loss'` | Two classes on one element | Right-aligned and red |

**Check yourself**
Every <tr> has the same number of cells; a missing <td> shifts the rest of the row left.

## Step 5. Style the table

**What I was thinking**
"Browsers draw tables with gaps between cells. border-collapse removes them, so lines are single."
"Stripes help the eye follow a row; nth-child(even) picks every second row without any extra classes."

```
table { border-collapse: collapse; width: 100%; font-size: 14px; }
th, td { padding: 8px 10px; border-bottom: 1px solid #e5e7eb; text-align: left; }
th { background: #ffe699; }
tbody tr:nth-child(even) { background: #f4f6fa; }   /* striped rows */
tbody tr:hover { background: #eef2f8; }             /* the row under the mouse */
.num  { text-align: right; }                        /* numbers line up on the right */
.loss { color: #b91c1c; }                           /* red for negatives */
```




**hello_table.html** *in the browser: header bar, heading, card, striped table, red negatives.*

| Tool | What it does | Why here |
|---|---|---|
| `border-collapse: collapse` | One line between cells, not two | Clean grid |
| `th, td { ... }` | One rule for two selectors | Comma means 'and' |
| `tbody tr:nth-child(even)` | Every second row in the body | Stripes, no extra classes |
| `:hover` | While the mouse is over it | Highlights the row you're reading |
| `text-align: right` | Right alignment | Decimal points line up |

**Check yourself**
Open hello_table.html: you should see the page below.

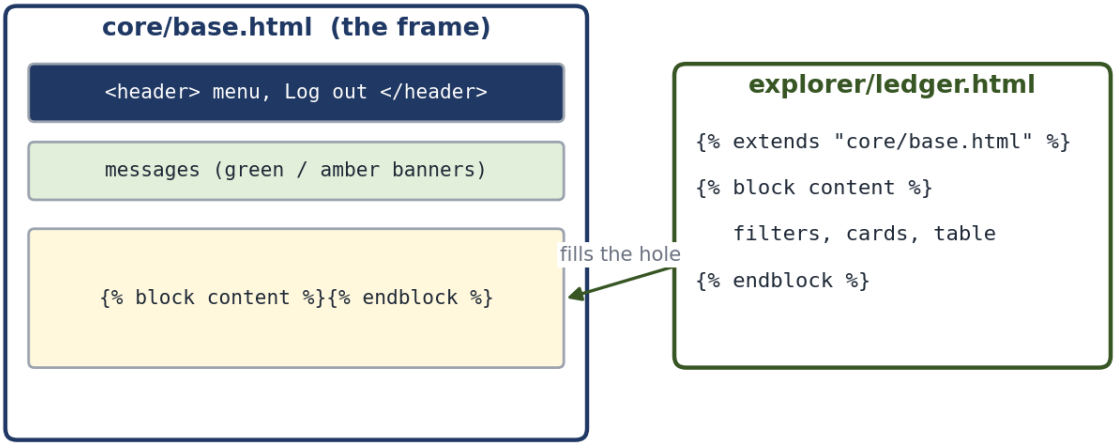
# Part 3. Join a page to your project

## Step 6. A template is HTML with holes

**What I was thinking**
"In Django, the HTML lives in a template. Two kinds of holes: {{ value }} prints something; {% tag %} does something (loop, if, link)."
"Every page repeats the header and styles. base.html holds them once; a page extends it and fills one block."

```
{% extends "core/base.html" %}                 {# use base.html as the frame #}
{% block title %}Ledger explorer{% endblock %}  {# the tab's text #}

{% block content %}                             {# everything inside goes into base.html's hole #}
<div class="card">
  <h2>Ledger explorer</h2>
  <p>Rows: {{ totals.rows }}</p>                {# prints a value from the view #}
  {% if totals.rows == 0 %}<p class="muted">Nothing found.</p>{% endif %}
</div>
{% endblock %}
```




*Every page keeps the same header and styles; it only writes what goes inside the block.*

| Tool | What it does | Why here |
|---|---|---|
| `{{ x }}` | Prints a value | Data from the view |
| `{{ x.y }}` | A key, attribute or index of x | row.debit, page_obj.number |
| `{{ x\|filter:'arg' }}` | Changes how a value prints | \|date, \|floatformat, \|slugify |
| `{% if %} ... {% endif %}` | Show only when true | Empty states, optional parts |
| `{% for %} ... {% endfor %}` | Repeat for each item | Table rows |
| `{% extends 'core/base.html' %}` | Use another template as the frame | Must be the first line |
| `{% block content %}` | A named hole to fill | base.html has the empty one |
| `{# ... #}` | A template comment | Never sent to the browser |

**Check yourself**
A template with no {% extends %} still works, but loses the header, menu and styles.

## Step 7. A new app, a view, a URL

**What I was thinking**
"A new page with its own job gets its own app, so it follows the same shape as commission and activity."
"Three things connect a page: a view (Python), a URL that calls it, and the project URL that includes the app."
"The explorer has no database table of its own, so it needs no models.py or migrations."

```
# terminal, in the project folder
python manage.py startapp explorer

# nexora_reports/settings.py
INSTALLED_APPS = [..., "core", "commission", "activity", "explorer"]

# explorer/views.py  (first version: just the page)
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def ledger(request):
    return render(request, "explorer/ledger.html", {})

# explorer/urls.py
from django.urls import path
from . import views

app_name = "explorer"
urlpatterns = [path("ledger/", views.ledger, name="ledger")]

# nexora_reports/urls.py
path("explorer/", include("explorer.urls")),

# the template file goes in:  explorer/templates/explorer/ledger.html
```

| Tool | What it does | Why here |
|---|---|---|
| `startapp explorer` | Creates the app folder | views.py, apps.py, tests.py ... |
| `INSTALLED_APPS` | Django's list of apps | Templates and tests are found only for listed apps |
| `render(request, template, {})` | Template + data -> HTML response | {} is the context: the data |
| `app_name = 'explorer'` | URL namespace | {% url 'explorer:ledger' %} |
| `include('explorer.urls')` | Hands /explorer/... to the app | The app never knows its prefix |
| `templates/explorer/ledger.html` | Folder named after the app | No clashes with other apps' templates |

**Check yourself**
Open /explorer/ledger/ after logging in: the page shows with the normal header and menu.

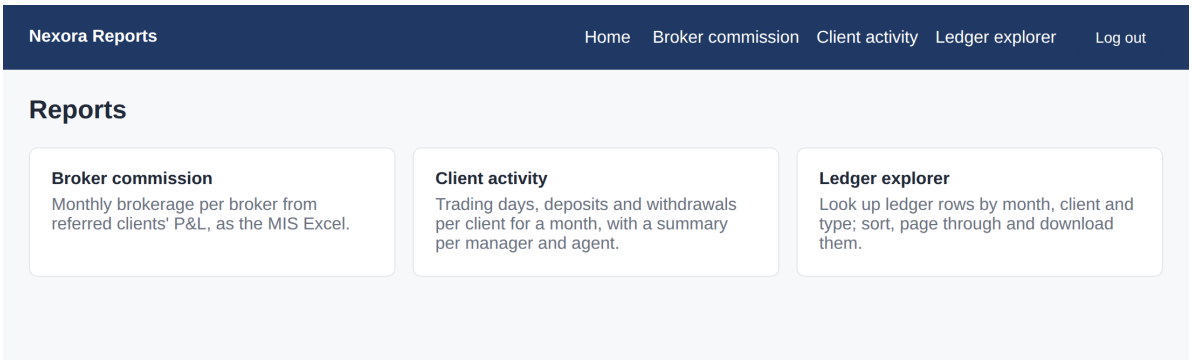
## Step 8. Put it in the menu

**What I was thinking**
"The menu builds itself from the report registry. One register() call in the app's ready() adds the page to the menu and the home page."

```
# explorer/apps.py
from django.apps import AppConfig


class ExplorerConfig(AppConfig):
    name = "explorer"
    verbose_name = "Ledger explorer"

    def ready(self):
        from core.reports import Report, register
        register(Report(key="explorer", title="Ledger explorer",
                        description="Look up ledger rows by month, client and type; sort, page through and "
                                    "download them.",
                        url_name="explorer:ledger"))
```




| Tool | What it does | Why here |
|---|---|---|
| `def ready(self)` | Runs once when Django starts | The moment to register |
| `import inside ready()` | Delays the import | Avoids AppRegistryNotReady |
| `url_name='explorer:ledger'` | Where the menu link points | Must exist in urls.py first |

**Check yourself**
Restart the server: 'Ledger explorer' appears in the header menu and as a card on the home page.

# Part 4. Put data in the table

## Step 9. One <tr> per row with {% for %}

**What I was thinking**
"The made-up rows from Part 2 become one row written once, inside a loop. The view sends a list; the template repeats the <tr> for each item."
"An empty list needs a message, not a blank table: {% empty %}."

```
<table class="data">
  <thead>
    <tr><th>Client ID</th><th>Date & time</th><th>Type</th><th class="num">Debit</th><th class="num">Credit</th></tr>
  </thead>
  <tbody>
    {% for row in page_obj %}
      <tr>
        <td>{{ row.client }}</td>
        <td>{{ row.created|date:"d M Y H:i" }}</td>
        <td><span class="pill type-{{ row.type|slugify }}">{{ row.type }}</span></td>
        <td class="num">{{ row.debit|floatformat:"2g" }}</td>
        <td class="num">{{ row.credit|floatformat:"2g" }}</td>
      </tr>
    {% empty %}
      <tr><td colspan="5" class="muted">No ledger rows match these filters.</td></tr>
    {% endfor %}
  </tbody>
</table>
```

| Tool | What it does | Why here |
|---|---|---|
| `{% for row in page_obj %}` | Repeats the block per row | page_obj is the list for this page |
| `{% empty %}` | Shown when the list has no items | A clear 'nothing found' row |
| `colspan='5'` | One cell spanning 5 columns | The message fills the row |
| `row.client` | The 'client' key of a dict | Dots work for dict keys without spaces |

**Check yourself**
Temporarily send page_obj = [] from the view: the table shows the 'No ledger rows' line.

## Step 10. A DataFrame becomes rows

**What I was thinking**
"Templates read dict keys with dots, and a dot can't handle a space or odd capitals. So rename the columns to short names first: 'Debit' becomes 'debit', 'ClientID' becomes 'client'."
"to_dict('records') turns the table into a list of dicts, one per row: exactly what {% for %} wants."

```
COLUMNS = {"client": "ClientID", "created": "CreatedAt", "type": "LedgerType", "debit": "Debit", "credit": "Credit"}

led, notes = get_ledger_slice(start, end, clients=..., types=...)
table = led.rename(columns={v: k for k, v in COLUMNS.items()})[list(COLUMNS)]
rows = table.to_dict("records")
# rows[0] -> {"client": "S200022", "created": Timestamp("2026-08-08 15:00"), "type": "WITHDRAW",
#             "debit": 743.0, "credit": 0.0}
```

| Tool | What it does | Why here |
|---|---|---|
| `{v: k for k, v in COLUMNS.items()}` | Flips the dict: ledger name -> short name | One mapping, used both ways |
| `df.rename(columns=...)` | Renames columns | 'Debit' -> 'debit' |
| `[list(COLUMNS)]` | Keeps these columns, in this order | The table's column order |
| `df.to_dict('records')` | A list of dicts, one per row | Ready for {% for row in ... %} |

**Check yourself**
print(rows[0]) in the view shows one dict with the five short keys.

## Step 11. Format values in the template

**What I was thinking**
"Keep the numbers as numbers in Python, and decide how they look in the template. Then sorting and totals still work on real numbers."

```
{{ row.debit|floatformat:"2g" }}           743.00      1,234,567.89     (2 decimals, thousands separators)
{{ row.created|date:"d M Y H:i" }}         08 Aug 2026 15:00
{{ row.type|slugify }}                     withdraw-cancelled    (safe for a CSS class name)

/* in base.html: one colour per ledger type */
.pill { display:inline-block; padding:2px 8px; border-radius:10px; font-size:11px; font-weight:bold; }
.pill.type-bill { background:#e2efda; color:#375623; }
.pill.type-withdraw, .pill.type-withdraw-cancelled { background:#fbe5d6; color:#843c0c; }
```

| Tool | What it does | Why here |
|---|---|---|
| `\|floatformat:'2g'` | 2 decimals; g adds thousands separators | Checked on your settings: 1,234,567.89 |
| `\|date:'d M Y H:i'` | Date in your format | d day, M month name, Y year, H:i time |
| `\|slugify` | Lower case, spaces to dashes | Turns a value into a CSS class name |
| `.pill.type-bill` | Both classes on one element | The BILL badge colour |

**Check yourself**
Change the date format to 'Y-m-d' and reload: dates show as 2026-08-08.

**The quick alternative: df.to_html()**
`table.to_html(index=False, classes='data')` plus `{{ html|safe }}` gives a table in one line; core.web.sheet_preview does exactly this. Use it for read-only previews. Use the loop when you need links, colours per value, badges, or sortable headings: the loop gives you control of every cell.
`|safe` tells Django the HTML is trusted. Only use it on HTML you built yourself, never on text a user typed.

# Part 5. Make the table usable

## Step 12. Filters with a GET form

**What I was thinking**
"Filters only read data, so they use method='get': the choices go into the address, and the page can be bookmarked or sent to a colleague."
"A Django form still checks them: a bad month shows an error instead of crashing."
"First visit has no month, so default it to last month before checking."

```
# explorer/forms.py
class LedgerFilterForm(forms.Form):
    month = forms.CharField(widget=MONTH_WIDGET)
    client = forms.CharField(required=False, label="Client ID")
    type = forms.ChoiceField(required=False, label="Ledger type",
                             choices=[("", "All types")] + [(t, t.title()) for t in LEDGER_TYPES])

    def clean_month(self):
        return clean_month_value(self.cleaned_data["month"])

    def clean_client(self):
        return self.cleaned_data["client"].strip().upper()

# explorer/views.py
data = request.GET.copy()
data.setdefault("month", previous_month())        # first visit: last month
form = LedgerFilterForm(data)
if form.is_valid():
    month = form.cleaned_data["month"]
    ...

<!-- template -->
<form method="get" class="filters">
  {{ form.month.label_tag }} {{ form.month }}
  {{ form.client.label_tag }} {{ form.client }}
  {{ form.type.label_tag }} {{ form.type }}
  <button class="btn" type="submit">Show</button>
</form>
{{ form.errors }}
```

| Tool | What it does | Why here |
|---|---|---|
| `forms.Form (not ModelForm)` | A form that saves nothing | Filters only |
| `ChoiceField(choices=[...])` | A dropdown | ('', 'All types') means no filter |
| `request.GET.copy()` | An editable copy of the address values | request.GET itself is read-only |
| `data.setdefault('month', ...)` | Adds month only if missing | A default for the first visit |
| `<form method='get'>` | Values go in the address | No {% csrf_token %} needed for GET |
| `{{ form.month.label_tag }} {{ form.month }}` | Draw one field yourself | Fields side by side, not as_p |

**Check yourself**
Type s200001 in Client ID: only that client's rows show (the form upper-cases it).

## Step 13. Click a heading to sort

> **What I was thinking**
> "The heading is a link to the same page with sort= changed. All other filters must stay, so build the link from the current address."
> "Never sort by whatever the address says: only by columns I know. Anything else falls back to newest first."

```
def url_with(request, **changes):
    """The current address with some values changed: url_with(request, sort='debit', page=None)."""
    params = request.GET.copy()
    for key, value in changes.items():
        if value is None:
            params.pop(key, None)              # None removes it (e.g. back to page 1)
        else:
            params[key] = value
    return "?" + params.urlencode()


sort = request.GET.get("sort", "-created")
key = sort.lstrip("-")
if key not in COLUMNS:                          # only columns we know
    sort, key = "-created", "created"
table = table.sort_values(key, ascending=not sort.startswith("-"), kind="stable")

for k, label, numeric in HEADERS:
    next_sort = "-" + k if sort == k else k    # click again to reverse
    arrow = "▲" if sort == k else "▼" if sort == "-" + k else ""
    headers.append({"label": label, "url": url_with(request, sort=next_sort, page=None),
                    "arrow": arrow, "numeric": numeric})

<!-- template -->
{% for h in headers %}
  <th{% if h.numeric %} class="num"{% endif %}><a href="{{ h.url }}">{{ h.label }} {{ h.arrow }}</a></th>
{% endfor %}
```




? starts the values, & joins them. In the view: request.GET.get('sort') gives '-debit'. url_with(request, page=3) rebuilds the address with one value changed and the rest kept.

| Tool | What it does | Why here |
|---|---|---|
| `params.urlencode()` | Back to 'month=2026-08&sort=debit' | Safe text for a link |
| `sort.lstrip('-')` | Removes a leading minus | '-debit' -> 'debit' |
| `if key not in COLUMNS` | A whitelist | The address can't pick an arbitrary column |
| `ascending=not sort.startswith('-')` | Minus means descending | -debit = largest first |
| `kind='stable'` | Ties keep their order | Clicking the same heading twice is predictable |
| `page=None` | Sorting goes back to page 1 | Page 7 of a new order makes no sense |

**Check yourself**
Click 'Debit' twice: the arrow flips and the order reverses; the month and client filters stay.

## Step 14. Pages of 50 rows

**What I was thinking**
"A month can have thousands of rows. Django's Paginator splits a list into pages and handles a bad page number."

```
from django.core.paginator import Paginator

page_obj = Paginator(table.to_dict("records"), 50).get_page(request.GET.get("page"))
prev_url = url_with(request, page=page_obj.previous_page_number()) if page_obj.has_previous() else None
next_url = url_with(request, page=page_obj.next_page_number()) if page_obj.has_next() else None

<!-- template -->
<span class="muted">Showing {{ page_obj.start_index }}–{{ page_obj.end_index }} of {{ totals.rows }} rows</span>
...
{% if page_obj.paginator.num_pages > 1 %}
  {% if prev_url %}<a class="btn secondary" href="{{ prev_url }}">‹ Previous</a>{% endif %}
  <span class="muted">Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}</span>
  {% if next_url %}<a class="btn secondary" href="{{ next_url }}">Next ›</a>{% endif %}
{% endif %}
```

| Tool | What it does | Why here |
|---|---|---|
| `Paginator(items, 50)` | Splits a list into pages of 50 | Works on any list |
| `.get_page(n)` | Page n; a bad or missing n gives page 1 or the last | No crash on page=abc |
| `page_obj.start_index / end_index` | Row numbers on this page | 'Showing 51-100' |
| `has_previous() / next_page_number()` | Is there a page before / which is next | Builds the two buttons |

**Check yourself**
Add &page=999 to the address: you get the last page, not an error.

## Step 15. Totals and a CSV download

**What I was thinking**
"The cards above the table use the whole filtered table, not just the 50 rows on screen."
"A download is a different response: instead of HTML, a CSV file. Same filters, same sort, every row."

```
totals = {"rows": len(table), "clients": table["client"].nunique(),
          "debit": table["debit"].sum(), "credit": table["credit"].sum()}

if request.GET.get("download") == "csv":
    response = HttpResponse(content_type="text/csv")
    response["Content-Disposition"] = f'attachment; filename="ledger_{month}.csv"'
    table.rename(columns=COLUMNS).to_csv(response, index=False)     # back to the ledger's column names
    return response

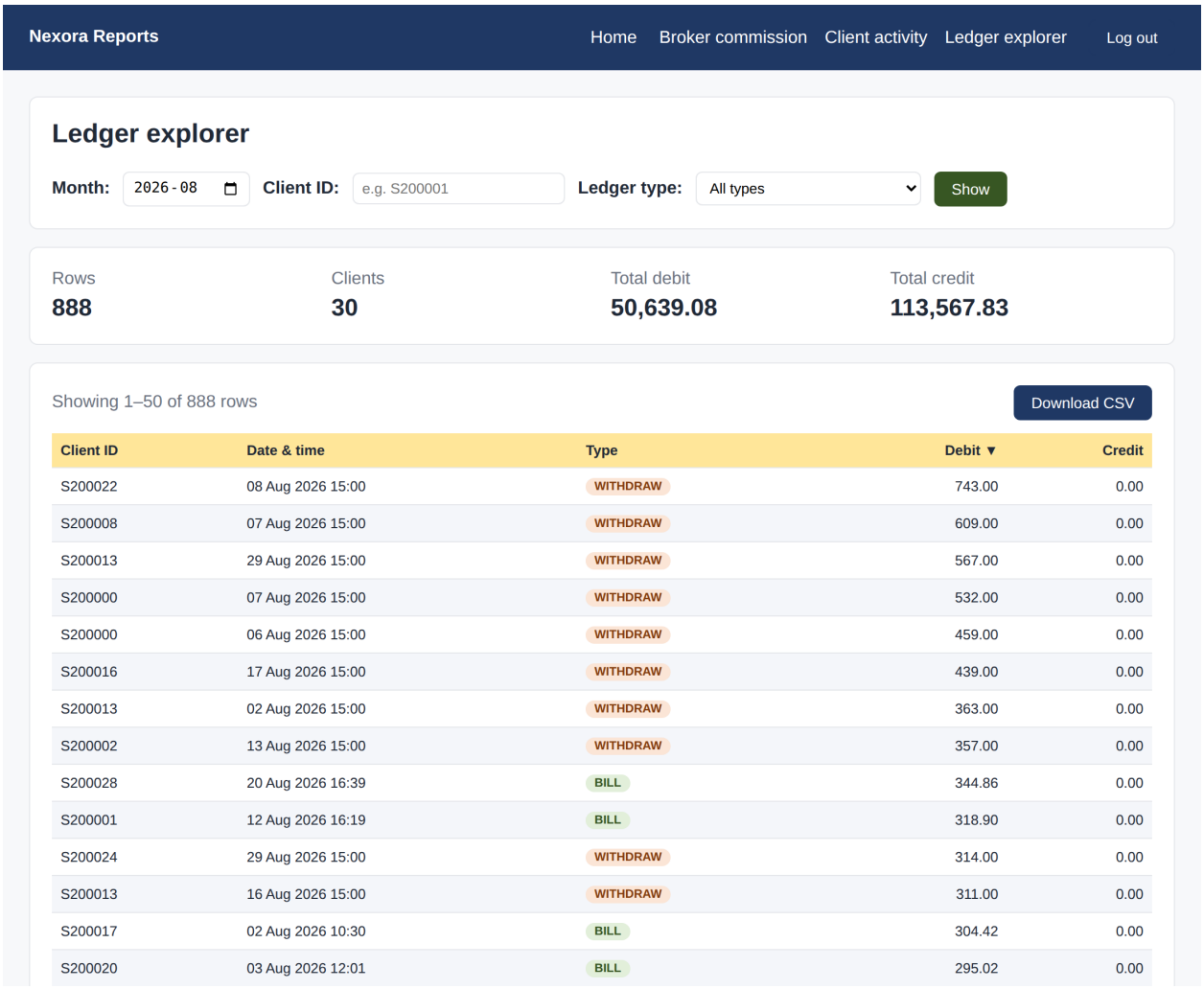
csv_url = url_with(request, download="csv", page=None)            # the button's link

<!-- template -->
<a class="btn secondary" href="{{ csv_url }}">Download CSV</a>
```

| Tool | What it does | Why here |
|---|---|---|
| `HttpResponse(content_type='text/csv')` | A response that is a file, not a page | The browser downloads it |
| `Content-Disposition: attachment` | Save, don't display | And the file name |
| `df.to_csv(response, ...)` | Writes straight into the response | HttpResponse behaves like a file |
| `download=csv in the address` | The same view, a different output | Filters and sort carry over |

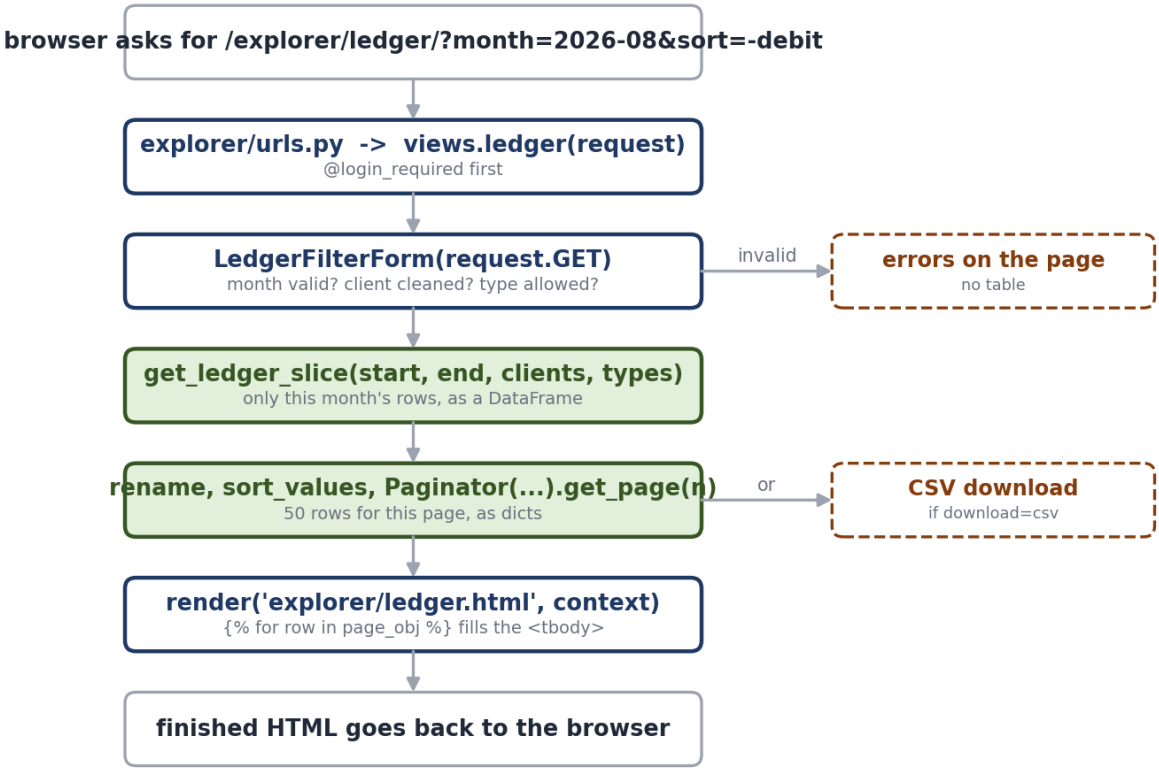
**Check yourself**
The CSV's row count equals the 'Rows' card, and its columns are ClientID, CreatedAt, LedgerType, Debit, Credit.

## The finished page




*August 2026 in the demo ledger, sorted by Debit (largest first), with the type badges, right-aligned numbers, totals and the download button.*

# Part 6. The whole request, start to finish




## Mistakes this page avoids

| If you wrote... | What would happen | What the page does instead |
|---|---|---|
| `{{ row.Debit Amount }}` | Template error: dots can't read keys with spaces | Rename columns to short names first |
| `<form method='post'> for filters` | No bookmarkable link; needs CSRF; Back button re-posts | method='get' |
| `<a href='?sort=debit'>` | Clicking a heading drops the month and client | url_with(request, sort=...) keeps them |
| `sort_values(request.GET['sort'])` | KeyError, or sorting by any column named in the address | A whitelist, with a default |
| `Formatting numbers as text in Python` | Sorting '1,000' before '200' as text | Numbers stay numbers; format in the template |
| `Totals from page_obj` | Totals of 50 rows, not the month | Totals from the full filtered table |
| `{{ html\|safe }} on user input` | Lets someone inject HTML or scripts | \|safe only on HTML you built |
| `A table with no {% empty %}` | A blank table looks like a bug | A clear 'No rows match' line |
| `Everything in one giant view` | Hard to test and change | Form, helper, view and template each do one job |

# Part 7. Checkpoints

These are tests from explorer/tests.py. With the demo ledger they all pass. Write each one before the step it checks, then make it pass.

```
from django.contrib.auth import get_user_model
from django.test import RequestFactory, TestCase, override_settings
from django.urls import reverse

from core.demo_ledger import DEMO_CLIENTS
from .views import url_with

URL = reverse("explorer:ledger")


@override_settings(LEDGER_MODULE="core.demo_ledger")
class LedgerExplorerTests(TestCase):
    def setUp(self):
        get_user_model().objects.create_user("analyst", password="pass12345")
        self.client.login(username="analyst", password="pass12345")

    def test_month_shows_a_table(self):                          # Steps 7 to 10
        response = self.client.get(URL, {"month": "2026-08"})
        self.assertContains(response, "<table")
        self.assertEqual(len(response.context["page_obj"]), 50)

    def test_client_filter(self):                                # Step 12
        client_id = DEMO_CLIENTS[0]
        rows = list(self.client.get(URL, {"month": "2026-08", "client": client_id.lower()}).context["page_obj"])
        self.assertTrue(rows and all(r["client"] == client_id for r in rows))

    def test_sort_by_debit_both_ways(self):                      # Step 13
        down = [r["debit"] for r in self.client.get(URL, {"month": "2026-08", "sort": "-debit"}).context["page_obj"]]
        up = [r["debit"] for r in self.client.get(URL, {"month": "2026-08", "sort": "debit"}).context["page_obj"]]
        self.assertEqual(down, sorted(down, reverse=True))
        self.assertEqual(up, sorted(up))

    def test_second_page(self):                                  # Step 14
        page = self.client.get(URL, {"month": "2026-08", "page": 2}).context["page_obj"]
        self.assertEqual((page.number, page.start_index()), (2, 51))

    def test_bad_month_shows_an_error(self):                     # Step 12
        response = self.client.get(URL, {"month": "08-2026"})
        self.assertContains(response, "Use the format YYYY-MM")


class UrlWithTests(TestCase):
    def test_keeps_filters_and_changes_one_value(self):          # Step 13
        request = RequestFactory().get("/explorer/ledger/", {"month": "2026-08", "client": "S1", "page": "3"})
        url = url_with(request, sort="debit", page=None)
        self.assertIn("month=2026-08", url)
        self.assertIn("sort=debit", url)
        self.assertNotIn("page=", url)
```

## Practice: your next page

1. **Colour the money:** give the Credit cell class='gain' when it's above 0, with a green colour in base.html. Hint: `{% if row.credit > 0 %}`.
2. **A totals row:** add a <tfoot> under <tbody> showing total debit and credit for the whole filter.
3. **A client link:** make each Client ID a link that filters the page to that client, using url_with in the view or a ?client= link in the template.
4. **A second page from scratch:** a 'Run history' page listing every commission and activity run in one table, newest first. Draw it, write it as plain HTML, then follow Steps 6 to 15.

## Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. What do the three layers (HTML, CSS, template) each do?
2. Why do header cells go in <thead> and data cells in <tbody>?
3. Why rename DataFrame columns before sending rows to the template?
4. Why do the filters use method='get' rather than 'post'?
5. What does url_with(request, sort='debit', page=None) return, and why page=None?
6. Why is there a whitelist for the sort column?
7. Why are the totals calculated from the table, not from page_obj?
8. When is {{ html|safe }} acceptable?

### Answers

1. HTML gives structure (what's on the page), CSS gives the look, the template fills in the data the view sends.
2. So CSS and browsers can treat them differently (gold headers, striped body) and screen readers know which is which.
3. Templates read dict keys with dots, which can't handle spaces or odd names; short names also make the template clearer.
4. Filters only read data: GET puts them in the address, so pages can be bookmarked, shared and refreshed safely.
5. '?month=...&client=...&sort=debit' with any page number removed: a new sort should start at page 1.
6. So the address can't sort by an unknown column (an error) or by anything you didn't intend to expose.
7. page_obj holds only the 50 rows on screen; the cards should describe every row that matches the filters.
8. Only for HTML your own code built (like df.to_html of your data); never for text a user typed.

# Appendix. The code, with line numbers

## hello_table.html (Part 2)

```
 1  <!doctype html>
 2  <html lang="en">
 3  <head>
 4    <meta charset="utf-8">
 5    <meta name="viewport" content="width=device-width, initial-scale=1">
 6    <title>My first table</title>
 7    <style>
 8      /* 1. The page */
 9      body { font-family: Arial, sans-serif; color: #1f2937; background: #f7f8fa; margin: 0; }
10      header { background: #1f3864; color: white; padding: 14px 24px; }
11      main { max-width: 900px; margin: 24px auto; padding: 0 24px; }
12
13      /* 2. A white box with a border: reused for anything we want to frame */
14      .card { background: white; border: 1px solid #e5e7eb; border-radius: 8px; padding: 20px; }
15
16      /* 3. The table */
17      table { border-collapse: collapse; width: 100%; font-size: 14px; }
18      th, td { padding: 8px 10px; border-bottom: 1px solid #e5e7eb; text-align: left; }
19      th { background: #ffe699; }
20      tbody tr:nth-child(even) { background: #f4f6fa; }   /* striped rows */
21      tbody tr:hover { background: #eef2f8; }             /* highlight the row under the mouse */
22      .num { text-align: right; }                         /* numbers line up on the right */
23      .loss { color: #b91c1c; }                           /* red for negative numbers */
24    </style>
25  </head>
26  <body>
27    <header>
28      <strong>Nexora Reports</strong>
29    </header>
30
31    <main>
32      <h1>Broker summary</h1>
33      <p>August 2026. Numbers are in USD.</p>
34
35      <div class="card">
36        <table>
37          <thead>
38            <tr>
39              <th>Broker</th>
40              <th>Clients</th>
41              <th class="num">Profit</th>
42              <th class="num">Loss</th>
43              <th class="num">Difference</th>
44            </tr>
45          </thead>
46          <tbody>
47            <tr>
48              <td>Ahmad Ali</td>
49              <td>12</td>
50              <td class="num">4,250.00</td>
51              <td class="num">9,800.50</td>
52              <td class="num loss">-5,550.50</td>
53            </tr>
54            <tr>
55              <td>Sara Khan</td>
56              <td>8</td>
57              <td class="num">7,100.00</td>
58              <td class="num">2,300.00</td>
59              <td class="num">4,800.00</td>
60            </tr>
61            <tr>
62              <td>Ravi Kumar</td>
```

```
63              <td>5</td>
64              <td class="num">950.25</td>
65              <td class="num">1,200.00</td>
66              <td class="num loss">-249.75</td>
67            </tr>
68          </tbody>
69        </table>
70      </div>
71
72      <p><a href="https://developer.mozilla.org/en-US/docs/Web/HTML/Element/table">Read more about HTML tables</a></p>
73    </main>
74  </body>
75  </html>
```

## explorer/forms.py

```
from django import forms

from core.forms import MONTH_WIDGET, clean_month_value

LEDGER_TYPES = ["BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED", "NEGATIVE BALANCE PROTECTION"]


class LedgerFilterForm(forms.Form):
    """The filters above the table. Sent with GET, so they appear in the address bar."""

    month = forms.CharField(widget=MONTH_WIDGET)
    client = forms.CharField(required=False, label="Client ID",
                             widget=forms.TextInput(attrs={"placeholder": "e.g. S200001"}))
    type = forms.ChoiceField(required=False, label="Ledger type",
                             choices=[("", "All types")] + [(t, t.title()) for t in LEDGER_TYPES])

    def clean_month(self):
        return clean_month_value(self.cleaned_data["month"])

    def clean_client(self):
        return self.cleaned_data["client"].strip().upper()
```

## explorer/views.py

```python
from django.contrib.auth.decorators import login_required
from django.core.paginator import Paginator
from django.http import HttpResponse
from django.shortcuts import render

from core.ledger import get_ledger_slice
from core.periods import month_range, previous_month

from .forms import LEDGER_TYPES, LedgerFilterForm

# Short names for the page (easy to use in templates) -> the ledger's own column names
COLUMNS = {"client": "ClientID", "created": "CreatedAt", "type": "LedgerType", "debit": "Debit", "credit": "Credit"}
HEADERS = [("client", "Client ID", False), ("created", "Date & time", False), ("type", "Type", False),
           ("debit", "Debit", True), ("credit", "Credit", True)]          # (key, label, is it a number?)
ROWS_PER_PAGE = 50


def url_with(request, **changes):
    """The current address with some query values changed: url_with(request, sort='debit', page=None)."""
    params = request.GET.copy()                # request.GET can't be changed; a copy can
    for key, value in changes.items():
        if value is None:
            params.pop(key, None)              # None removes the value (e.g. back to page 1)
        else:
            params[key] = value
    return "?" + params.urlencode()


@login_required
def ledger(request):
    data = request.GET.copy()
    data.setdefault("month", previous_month())  # first visit: last month
    form = LedgerFilterForm(data)
    context = {"form": form}

    if form.is_valid():
        month, client, ltype = (form.cleaned_data[k] for k in ("month", "client", "type"))
        start, end = month_range(month)
        led, notes = get_ledger_slice(start, end, clients=[client] if client else None,
                                      types=[ltype] if ltype else LEDGER_TYPES)

        # Short column names, then sort by the column the user clicked
        table = led.rename(columns={v: k for k, v in COLUMNS.items()})[list(COLUMNS)]
        sort = request.GET.get("sort", "-created")
        key = sort.lstrip("-")
        if key not in COLUMNS:                 # only sort by columns we know about
            sort, key = "-created", "created"
        table = table.sort_values(key, ascending=not sort.startswith("-"), kind="stable")

        if request.GET.get("download") == "csv":
            response = HttpResponse(content_type="text/csv")
            response["Content-Disposition"] = f'attachment; filename="ledger_{month}.csv"'
            table.rename(columns=COLUMNS).to_csv(response, index=False)
            return response

        headers = []
        for k, label, numeric in HEADERS:
            next_sort = "-" + k if sort == k else k            # click again to reverse
            arrow = "▲" if sort == k else "▼" if sort == "-" + k else ""
            headers.append({"label": label, "url": url_with(request, sort=next_sort, page=None),
                            "arrow": arrow, "numeric": numeric})

        page_obj = Paginator(table.to_dict("records"), ROWS_PER_PAGE).get_page(request.GET.get("page"))
        context.update({
            "month": month, "notes": notes, "headers": headers, "page_obj": page_obj,
            "totals": {"rows": len(table), "clients": table["client"].nunique(),
                       "debit": table["debit"].sum(), "credit": table["credit"].sum()},
            "csv_url": url_with(request, download="csv", page=None),
            "prev_url": url_with(request, page=page_obj.previous_page_number()) if page_obj.has_previous() else None,
            "next_url": url_with(request, page=page_obj.next_page_number()) if page_obj.has_next() else None,
        })
    return render(request, "explorer/ledger.html", context)
```

| Lines | Block | Purpose |
|---|---|---|
| 12-15 | `Constants` | Short names, headings, page size |
| 18-26 | `url_with()` | Links that keep the filters |
| 30-34 | `Form` | Read and check the filters |
| 36-48 | `Data + sort` | Slice, rename, sort |
| 50-54 | `CSV` | Download every filtered row |
| 56-72 | `Headers + pages` | Sort links, Paginator, context |

## explorer/urls.py

```
 1  from django.urls import path
 2
 3  from . import views
 4
 5  app_name = "explorer"
 6  urlpatterns = [path("ledger/", views.ledger, name="ledger")]
```

## explorer/apps.py

```
 1  from django.apps import AppConfig
 2
 3
 4  class ExplorerConfig(AppConfig):
 5      name = "explorer"
 6      verbose_name = "Ledger explorer"
 7
 8      def ready(self):
 9          from core.reports import Report, register
10          register(Report(key="explorer", title="Ledger explorer",
11                          description="Look up ledger rows by month, client and type; sort, page through and "
12                                      "download them.",
13                          url_name="explorer:ledger"))
```

## explorer/templates/explorer/ledger.html

```
{% extends "core/base.html" %}
{% block title %}Ledger explorer{% endblock %}

{% block content %}
<div class="card">
  <h2 style="margin-top:0">Ledger explorer</h2>

  {# The filters. method="get" puts them in the address, so the page can be bookmarked and shared. #}
  <form method="get" class="filters">
    {{ form.month.label_tag }} {{ form.month }}
    {{ form.client.label_tag }} {{ form.client }}
    {{ form.type.label_tag }} {{ form.type }}
    <button class="btn" type="submit">Show</button>
  </form>
  {{ form.errors }}
</div>

{% if page_obj %}
  {% for note in notes %}<div class="msg warning">{{ note }}</div>{% endfor %}

  <div class="card">
    <div class="stats">
      <div class="stat"><span class="muted">Rows</span><b>{{ totals.rows }}</b></div>
      <div class="stat"><span class="muted">Clients</span><b>{{ totals.clients }}</b></div>
      <div class="stat"><span class="muted">Total debit</span><b>{{ totals.debit|floatformat:"2g" }}</b></div>
      <div class="stat"><span class="muted">Total credit</span><b>{{ totals.credit|floatformat:"2g" }}</b></div>
    </div>
  </div>

  <div class="card">
    <div class="toolbar">
      <span class="muted">Showing {{ page_obj.start_index }}–{{ page_obj.end_index }} of {{ totals.rows }} rows</span>
      <a class="btn secondary" href="{{ csv_url }}">Download CSV</a>
    </div>
    <div class="scroll">
      <table class="data">
        <thead>
          <tr>
            {% for h in headers %}
              <th{% if h.numeric %} class="num"{% endif %}><a href="{{ h.url }}">{{ h.label }} {{ h.arrow }}</a></th>
            {% endfor %}
          </tr>
        </thead>
        <tbody>
          {% for row in page_obj %}
            <tr>
              <td>{{ row.client }}</td>
              <td>{{ row.created|date:"d M Y H:i" }}</td>
              <td><span class="pill type-{{ row.type|slugify }}">{{ row.type }}</span></td>
              <td class="num">{{ row.debit|floatformat:"2g" }}</td>
              <td class="num">{{ row.credit|floatformat:"2g" }}</td>
            </tr>
          {% empty %}
            <tr><td colspan="5" class="muted">No ledger rows match these filters.</td></tr>
          {% endfor %}
        </tbody>
      </table>
    </div>

    {% if page_obj.paginator.num_pages > 1 %}
      <div class="pager">
        {% if prev_url %}<a class="btn secondary" href="{{ prev_url }}">‹ Previous</a>{% endif %}
        <span class="muted">Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}</span>
        {% if next_url %}<a class="btn secondary" href="{{ next_url }}">Next ›</a>{% endif %}
      </div>
    {% endif %}
  </div>
{% endif %}
{% endblock %}
```